



UNIDADE CURRICULAR: Compilação

CÓDIGO: 21018

DOCENTE: Constantino e Rudi

Grupo: QUADCORE

João Rodrigues (2203474)

Maria Costa (2304361)

Nuno Rolo (1900405)

Fábio Oliveira (1800960)

CURSO: Licenciatura em Engenharia Informática

DATA DE ENTREGA: 22/06/2026

RELATÓRIO

1. INTRODUÇÃO

O presente trabalho corresponde à versão final do compilador para a linguagem MOCP desenvolvido ao longo da unidade curricular de Compilação.

A versão submetida integra todas as funcionalidades implementadas nos e-Fólios A e B, bem como um conjunto de melhorias introduzidas após o feedback obtido na avaliação do e-Fólio B. Foi ainda acrescentada a geração de código final MIPS32, completando assim a cadeia clássica de compilação.

O compilador foi desenvolvido em Python e utiliza ANTLR4 para análise léxica e sintática.

2. ARQUITETURA DO COMPILADOR

O compilador encontra-se organizado nas seguintes fases:

1. Análise léxica;
2. Análise sintática;
3. Construção da AST;
4. Análise semântica;
5. Geração de código intermédio TAC;
6. Otimização de TAC;
7. Geração de código final MIPS32.

Esta organização segue a arquitetura clássica apresentada na bibliografia da unidade curricular.

3. MELHORIAS RELATIVAMENTE AO E-FÓLIO B

Após a avaliação do e-Fólio B foram introduzidas diversas melhorias técnicas.

3.1 Curto-circuito em operadores lógicos

No e-Fólio B os operadores lógicos && e || eram avaliados através da combinação dos resultados dos operandos.

Na versão final foi implementada avaliação com curto-circuito recorrendo a labels e saltos condicionais, aproximando o comportamento ao das linguagens imperativas tradicionais.

3.2 Receção explícita de parâmetros

Foi introduzida a instrução TAC:

getparam índice

Esta alteração torna a convenção de chamadas explícita e torna o TAC mais auto-contido.

3.3 Inicialização explícita de vetores

Foi criada a pseudo-instrução:

array_zero_init vetor, tamanho

permitindo representar explicitamente a inicialização automática de vetores.

3.4 Conversões explícitas de tipo

Foram introduzidas operações intermédias:

`int_to_real`

`real_to_int`

tornando explícitas as conversões numéricas.

3.5 Validação de retornos

Foi reforçada a análise semântica para verificar que funções com tipo de retorno diferente de vazio possuem retorno válido em todos os caminhos relevantes de execução.

Estas melhorias foram implementadas em resposta direta às observações efetuadas na avaliação do e-Fólio B, procurando tornar o TAC mais auto-contido, semanticamente mais fiel ao comportamento da linguagem MOCP e mais adequado para servir de base à geração de código final.

4. OTIMIZAÇÃO DE TAC

O módulo de otimização implementa as seguintes técnicas:

Constant Folding

Exemplo:

`t1 = 3 * 4`

é transformado em:

`t1 = 12`

Simplificação Algébrica

Exemplo:

`x = y + 0`

é transformado em:

`x = y`

Constant Propagation

Exemplo:

`a = 5`

`b = a`

é transformado em:

`b = 5`

Copy Propagation

Exemplo:

`a = b`

`c = a`

é transformado em:

`c = b`

Dead Code Elimination

Temporários não utilizados são removidos de forma conservadora.

Foram ainda testados programas contendo curto-circuito lógico, passagem de parâmetros através de getparam, inicialização automática de vetores e geração de código MIPS para funções recursivas.

5. GERAÇÃO DE CÓDIGO FINAL

A principal evolução relativamente ao e-Fólio B consiste na geração de código final MIPS32.

Foi desenvolvido o módulo:

codegen_mips.py

responsável pela tradução do TAC otimizado para código assembly MIPS32. Esta abordagem permite separar claramente as fases de análise, otimização e geração de código, seguindo a arquitetura clássica dos compiladores apresentada na bibliografia da unidade curricular.

A implementação suporta:

- variáveis escalares;
- vetores;
- operações aritméticas;
- comparações;
- saltos condicionais;
- ciclos;
- chamadas de funções;
- recursividade;
- escrita em consola;
- conversões numéricas.

O código assembly MIPS32 gerado foi validado através do simulador MARS 4.5. As Figuras 2 e 3 apresentam, respetivamente, um excerto do código MIPS32 produzido automaticamente pelo compilador e a sua execução bem-sucedida no simulador.

Exemplo simplificado:

TAC:

a = 5

b = 3

t1 = a + b

MIPS:

li \$t0, 5

li \$t1, 3

add \$t2, \$t0, \$t1

Este exemplo ilustra a tradução direta de operações aritméticas do TAC para instruções MIPS32.

6. TESTES

Foi desenvolvida uma bateria de testes automática através do ficheiro:

run_tests.py

Os testes incluem:

- programas válidos;
- erros léxicos;
- erros sintáticos;
- erros semânticos;
- funções;
- recursividade;
- vetores;
- ciclos;
- otimizações;
- geração de código MIPS.

Todos os testes executados foram concluídos com sucesso.

7. CONCLUSÃO

O projeto atingiu o objetivo proposto pela unidade curricular, implementando um compilador completo para a linguagem MOCP.

A versão final integra análise léxica, análise sintática, análise semântica, geração de TAC, otimização de código intermédio e geração de código final MIPS32.

As melhorias introduzidas após o e-Fólio B permitiram tornar o compilador mais robusto, mais fiel à semântica da linguagem e mais próximo da arquitetura utilizada em compiladores reais.

8. BIBLIOGRAFIA

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson.
- Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.
- Material pedagógico da UC 21018 - Compilação, Universidade Aberta.

ANEXOS

Figura 1 – Exemplo de TAC gerado para o programa spec_fatorial.mocp, evidenciando a receção explícita de parâmetros através de getparam, a inicialização automática de variáveis e a otimização da passagem de parâmetros.

```
PS C:\Users\mffcf\MEOCLOUD\LEI\3 Ano\2Semestre\21018 Compilação\efolio B\efolio8_QUADCORE> python main.py Testes/spec_fatorial.mocp
Análise concluída com sucesso.
```

```
AST gerada:
```

```
-----
ProgramNode
  prototypes: [
    PrototypeNode
```

(...)

Código intermédio TAC otimizado:

```
-----
0000: func fact:
0001: declare k
0002: k = getparam 0
0003: t1 = k <= 1
0004: ifFalse t1 goto L1
0005: return 1
0006: goto L2
0007: L1:
0008: t2 = k - 1
0009: param t2
0010: t3 = call fact, 1
0011: t4 = k * t3
0012: return t4
0013: L2:
0014: endfunc fact
0015: func principal:
0016: declare n
0017: n = 0
0018: call escrevers, "Introduza inteiro: "
0019: t5 = call ler, 0
0020: n = t5
0021: param t5
0022: t6 = call fact, 1
0023: call escrever, t6
0024: endfunc principal
```

Código intermédio TAC original:

```
-----
0000: func fact:
0001: declare k
0002: k = getparam 0
0003: t1 = k <= 1
0004: ifFalse t1 goto L1
0005: return 1
0006: goto L2
0007: L1:
0008: t2 = k - 1
0009: param t2
0010: t3 = call fact, 1
0011: t4 = k * t3
0012: return t4
0013: L2:
0014: endfunc fact
0015: func principal:
0016: declare n
0017: n = 0
0018: call escrevers, "Introduza inteiro: "
0019: t5 = call ler, 0
0020: n = t5
0021: param n
0022: t6 = call fact, 1
0023: call escrever, t6
0024: endfunc principal
```

Figura 2 – Excerto do código assembly MIPS32 gerado automaticamente a partir do TAC otimizado para o programa spec_fatorial.mocp.

```

ASM spec_fatorial.asm X
Testes > ASM spec_fatorial.asm
1 # =====
2 # Segmento de dados
3 # =====
4 .data
5 str_0: .asciiz "Introduza inteiro: "
6
7 # =====
8 # Segmento de código
9 # =====
10 .text
11 .globl main
12
13 main:
14     jal f_principal
15     li $v0, 10
16     syscall
17
18 # ---- função fact (bastidor = 32 bytes) ----
19 f_fact:
20     addiu $sp, $sp, -32
21     sw $ra, 28($sp)
22     sw $fp, 24($sp)
23     addiu $fp, $sp, 32
24     lw $t2, 0($fp)
25     sw $t2, -32($fp)
26     # t1 = k <= 1
27     lw $t0, -32($fp)
28     li $t1, 1
29     sle $t2, $t0, $t1
30     sw $t2, -28($fp)
31     lw $t0, -28($fp)
32     beq $t0, $zero, L1
33     li $v0, 1
34     lw $ra, -4($fp)
35     lw $t8, -8($fp)
36     move $sp, $fp

```

stes/spec_fatorial.asmcloud\LEI\3 Ano\2Semestre\21018 Compilação\efolio B\efolioB_QUADCORE>
 Código MIPS escrito em: Testes/spec_fatorial.asm
 PS C:\Users\vmffcf\MEQCloud\LEI\3 Ano\2Semestre\21018 Compilação\efolio B\efolioB_QUADCORE> code Testes/spec_fatorial.asm
 PS C:\Users\vmffcf\MEQCloud\LEI\3 Ano\2Semestre\21018 Compilação\efolio B\efolioB_QUADCORE>

(...)

```

ASM spec_fatorial.asm X
Testes > ASM spec_fatorial.asm
73 f_principal:
76     sw $fp, 16($sp)
77     addiu $fp, $sp, 24
78     li $t0, 0
79     sw $t0, -24($fp)
80     # call escrevers, "Introduza inteiro: "
81     la $a0, str_0
82     li $v0, 4
83     syscall
84     # t5 = call ler, 0
85     li $v0, 5
86     syscall
87     sw $v0, -20($fp)
88     lw $t0, -20($fp)
89     sw $t0, -24($fp)
90     # t6 = call fact, 1
91     addiu $sp, $sp, -4
92     lw $t0, -20($fp)
93     sw $t0, 0($sp)
94     jal f_fact
95     addiu $sp, $sp, 4
96     sw $v0, -16($fp)
97     # call escrever, t6
98     lw $a0, -16($fp)
99     li $v0, 1
100    syscall
101    li $a0, 10
102    li $v0, 11
103    syscall
104 f_principal_end:
105     lw $ra, -4($fp)
106     lw $t8, -8($fp)
107     move $sp, $fp
108     move $fp, $t8
109     jr $ra
110
111

```

Figura 3 – Execução do código MIPS32 no simulador MARS 4.5, comprovando a correta tradução e execução do programa.

